# عنوان البحث

# JavaScript


## إعداد / إسراء محمد رمضان الدقاق

## فرقة / ثانية

## سكشن / ١٦

## مجموعة / أ

## الرقم الجامعي / #2500734

# المقدمة

أصبحت مواقع الويب جزءًا أساسيًا من حياتنا اليومية، حيث نستخدمها في التعليم والتسوق والتواصل الاجتماعي ومشاهدة الفيديوهات والبحث عن المعلومات. ومع التطور المستمر في التكنولوجيا، لم تعد المواقع مجرد صفحات تحتوي على نصوص وصور ثابتة، بل أصبحت صفحات تفاعلية تستجيب لحركات المستخدم وضغطاته وتساعده على إنجاز المهام بسهولة وسرعة.

فعندما يضغط المستخدم على زر تسجيل الدخول، أو يكتب اسمه في نموذج إلكتروني، أو يختار منتجًا من متجر إلكتروني، فإن الموقع يحتاج إلى وسيلة لفهم هذا التصرف وتنفيذ الأمر المناسب. وهنا يأتي دور لغة البرمجةJavaScript ، التي تُستخدم لإضافة الحركة والتفاعل إلى صفحات الويب.

تُعد JavaScript من أهم لغات البرمجة المستخدمة في تطوير المواقع الحديثة، لأنها تسمح للمبرمج بكتابة أوامر تجعل الصفحة تستجيب لأحداث مختلفة، مثل الضغط بالماوس، والكتابة باستخدام لوحة المفاتيح، وإرسال النماذج، وتحريك المؤشر فوق العناصر.

ولا يقتصر دور JavaScript على الاستجابة للضغطات فقط، بل يمكن استخدامها لتغيير محتوى الصفحة، وإظهار الرسائل، والتحقق من صحة البيانات، وتحديث بعض أجزاء الموقع دون الحاجة إلى إعادة تحميل الصفحة بالكامل.

## مشكلة البحث

تتمثل مشكلة البحث في فهم الطريقة التي تستطيع بها مواقع الويب اكتشاف تصرفات المستخدم والتفاعل معها، وكيف تُستخدم JavaScript لتنفيذ الأوامر المناسبة عند حدوث هذه التصرفات.

## أهداف البحث

١. التعرف على مفهوم لغة JavaScript وأهميتها.
٢. توضيح العلاقة بين HTML و CSSوJavaScript.
٣. فهم معنى الأحداث التي تحدث داخل صفحات الويب.
٤. التعرف على طريقة التعامل مع ضغطات الماوس ولوحة المفاتيح.
٥. شرح كيفية استخدام JavaScript في النماذج الإلكترونية.
٦. تقديم أمثلة عملية مبسطة على التفاعل داخل المواقع.
٧. التعرف على أهمية التفاعل في تحسين تجربة المستخدم.

## أهمية البحث

تنبع أهمية هذا البحث من أن التفاعل أصبح عنصرًا أساسيًا في تصميم مواقع الويب الحديثة. فالموقع الجيد لا يكتفي بعرض المعلومات، بل يساعد المستخدم على التعامل معها بسهولة، ويوفر له استجابة واضحة عند تنفيذ أي أمر.

كما يساعد فهم JavaScript المبتدئين على الانتقال من مجرد إنشاء صفحات ثابتة إلى بناء صفحات أكثر تطورًا وقدرة على الاستجابة لاحتياجات المستخدمين.

# تعريف JavaScript

JavaScriptهي لغة برمجة تُستخدم على نطاق واسع في تطوير مواقع الويب، وتساعد على جعل صفحات الإنترنت أكثر تفاعلًا وديناميكية. فعن طريقها يستطيع المبرمج تحديد ما يحدث عندما ينفذ المستخدم أمرًا معينًا، مثل الضغط على زر أو إدخال كلمة في مربع البحث.
على سبيل المثال، إذا كان هناك زر مكتوب عليه «اعرف المزيد»، فيمكن استخدام JavaScriptلجعل الموقع يعرض معلومات إضافية عند الضغط على هذا الزر. ويمكن أيضًا استخدامها لإظهار قائمة، أو تغيير صورة، أو عرض رسالة تؤكد نجاح عملية التسجيل.
وتعمل JavaScript في متصفحات الويب الحديثة، مثل Google Chrome و Microsoft Edge وMozilla Firefox، كما يمكن استخدامها في مجالات أخرى من البرمجة، مثل تطوير تطبيقات الخوادم باستخدام بيئات تشغيل مناسبة.

# الفرق بين HTML و CSSوJavaScript

تعتمد صفحات الويب غالبًا على ثلاثة عناصر رئيسية تعمل معًا، وهي HTML و CSS وJavaScript.

## أولًا: لغةHTML

تُستخدم HTML لتحديد بنية الصفحة ومحتوياتها، مثل العناوين والفقرات والصور والأزرار والروابط. ويمكن تشبيهها بالهيكل الأساسي الذي تُبنى عليه الصفحة.

## ثانيًا: لغةCSS

تُستخدم CSS لتنسيق شكل الصفحة، مثل الألوان والخطوط والأحجام والمسافات وترتيب العناصر. وهي المسؤولة عن جعل الصفحة جذابة ومنظمة من الناحية البصرية.

## ثالثًا: لغةJavaScript

تُستخدم JavaScript لإضافة السلوك والتفاعل. فهي التي تحدد ما الذي سيحدث عند ضغط المستخدم على زر، أو إدخال بيانات، أو تغيير اختيار معين.
وللتوضيح، إذا أنشأنا زرًا داخل صفحة ويب، فإن HTML تنشئ الزر، و CSSتحدد لونه وحجمه، بينما تستطيع JavaScript تحديد الإجراء الذي يحدث عند الضغط عليه.

## مثال مبسط

يمكن إنشاء زر بسيط باستخدام HTML بالشكل التالي:

```
<button id="myButton">اضغط هنا</button>
<p id="message"></p>
```

ثم نستخدم JavaScript لجعل الزر يعرض رسالة:

```
document.getElementById("myButton")
.addEventListener("click", function() {
document.getElementById("message").textContent =
"أهلًا بك في الموقع!";
});
```

عندما يضغط المستخدم على الزر، تظهر الرسالة أسفله. وهذا المثال يوضح كيف يمكن تحويل عنصر ثابت إلى عنصر تفاعلي باستخدام بضعة أسطر من التعليمات البرمجية.

# ما المقصود بالأحداث في JavaScript ؟

## تعريف الأحداث

الأحداث (Events) هي التصرفات أو التغييرات التي تحدث أثناء استخدام الموقع، ويستطيع المتصفح اكتشافها وإبلاغ البرنامج بها حتى ينفذ الإجراء المناسب.
وقد تنتج الأحداث عن تفاعل المستخدم مع الصفحة، أو عن تغييرات أخرى تحدث داخل المتصفح.
ومن أمثلتها الضغط على زر، وتحريك مؤشر الماوس، والكتابة في حقل نصي، وإرسال نموذج، وانتهاء تحميل الصفحة.
وتُعد الأحداث الأساس الذي تعتمد عليه معظم صور التفاعل فيJavaScript ، لأن المبرمج يستطيع ربط كل حدث بوظيفة معينة تُنفذ عند حدوثه.

## أهم أنواع الأحداث

### ١. حدث الضغطClick

يحدث عندما يضغط المستخدم على عنصر، مثل زر أو رابط. ويمكن استخدامه لإظهار رسالة أو فتح قائمة أو تنفيذ عملية حسابية.

### ٢. حدث تحريك المؤشرMouseover

يحدث عندما يتحرك مؤشر الماوس فوق عنصر معين. ويمكن استخدامه لتغيير لون الزر أو إظهار معلومات إضافية.

### ٣. حدث مغادرة المؤشرMouseout

يحدث عندما يغادر المؤشر العنصر، ويمكن استخدامه لإعادة العنصر إلى شكله الأصلي.

### ٤. أحداث لوحة المفاتيح

تشمل أحداثًا مثل keydownعند الضغط على أحد المفاتيح، و keyupعند ترك المفتاح بعد الضغط عليه. وتفيد هذه الأحداث في إنشاء اختصارات لوحة المفاتيح والتعامل مع إدخال البيانات.

### ٥. حدث الكتابةInput

يُستخدم لمتابعة التغييرات التي تحدث في قيمة حقل الإدخال، مثل كتابة اسم أو كلمة بحث.

### ٦. حدث إرسال النموذجSubmit

يحدث عند محاولة إرسال نموذج إلكتروني، مثل نموذج تسجيل الدخول أو التسجيل في موقع.

### ٧. حدث تحميل الصفحةLoad

يرتبط باكتمال تحميل المورد أو الصفحة حسب العنصر الذي رُبط به الحدث، ويمكن الاستفادة منه لتنفيذ بعض الإجراءات بعد اكتمال التحميل.

## الفرق بين الحدث ومعالج الحدث

الحدث هو التصرف الذي يقع، أما معالج الحدث (Event Handler) فهو الكود الذي يُنفذ استجابةً لهذا التصرف.
فعندما يضغط المستخدم على زر، يكون الضغط هو الحدث، بينما تكون الوظيفة التي تعرض رسالة ترحيب هي معالج الحدث.
ويمكن للمبرمج تحديد أكثر من معالج للحدث نفسه باستخدام addEventListener()، كما يمكن إزالة بعض مستمعي الأحداث عند عدم الحاجة إليهم.
وبذلك توفر الأحداث وسيلة منظمة تجعل البرنامج يستجيب للمستخدم بدلًا من تنفيذ الأوامر بطريقة ثابتة لا تتغير.

## كيف تتعرف JavaScript على ضغطات المستخدم؟

### استخدام مستمعات الأحداث

تحتاج JavaScript إلى معرفة العنصر الذي سيستقبل التفاعل، ثم تحديد الحدث المطلوب الاستجابة له، وبعد ذلك كتابة الكود الذي سينفذ عند وقوع الحدث.

ومن أشهر الطرق المستخدمة لذلك الطريقة addEventListener()، التي تسمح بربط عنصر معين بحدث محدد ووظيفة تُنفذ عند وقوعه.

وتتكون العملية من ثلاث خطوات أساسية:

١. الوصول إلى العنصر المطلوب داخل الصفحة.
٢. تسجيل مستمع للحدث الذي نريد اكتشافه.
٣. كتابة التعليمات التي ستنفذ عند حدوث الحدث.

### مثال على الضغط على زر

```
<button id="helloButton">اضغط هنا</button>
const button = document.getElementById("helloButton");

button.addEventListener("click", function() {
  alert("لقد ضغطت على الزر!");
});
```

في هذا المثال، يبحث البرنامج عن الزر الذي يحمل المعرّف helloButton، ثم يربط به حدث الضغط. وعندما يضغط المستخدم على الزر، تظهر نافذة تحتوي على رسالة.

### شرح الكود

- document.getElementById() تستخدم للوصول إلى عنصر يحمل معرّفًا محددًا.
- addEventListener() تُستخدم لتسجيل مستمع للحدث.
- click هو اسم الحدث الذي نريد مراقبته.
- function() تحتوي على التعليمات التي ستنفذ عند وقوع الحدث.
- alert() تعرض نافذة تنبيه للمستخدم.

### لماذا تُعد هذه الطريقة مهمة؟

تسمح هذه الطريقة بفصل التعليمات البرمجية عن عناصر الصفحة، كما تتيح تسجيل أكثر من مستمع للحدث نفسه عند الحاجة.

وتوفر أيضًا إمكانية إزالة مستمع لم يعد مطلوبًا باستخدام removeEventListener()، وهو أمر مفيد في التطبيقات الكبيرة التي تحتوي على عناصر تفاعلية كثيرة.

ومن المهم معرفة أن JavaScript لا تحتاج إلى كتابة وظيفة مستقلة لكل ضغطة يقوم بها المستخدم؛ بل يتم تسجيل مستمع الحدث مرة واحدة، ثم يستدعي المتصفح الوظيفة المناسبة كلما وقع الحدث المسجل.

وهكذا يمكن للبرنامج متابعة التفاعل وتنفيذ الأوامر في الوقت المناسب، دون الحاجة إلى إعادة كتابة الكود مع كل ضغطة.

## التفاعل مع الماوس ولوحة المفاتيح

### أولًا: التفاعل مع الماوس

يُستخدم الماوس في تنفيذ عدد كبير من العمليات داخل مواقع الويب، مثل الضغط على الأزرار، وفتح الروابط، واختيار العناصر، وتحريك المؤشر فوق الصور والقوائم.

وتستطيع JavaScript الاستجابة لهذه التصرفات عن طريق الأحداث المناسبة.

فعلى سبيل المثال، يمكن تغيير لون زر عندما يضغط المستخدم عليه، ثم إعادة اللون السابق عند الضغط مرة أخرى.

```
<button id="colorButton">تغيير اللون</button>
const button = document.getElementById("colorButton");

button.addEventListener("click", function() {
button.style.backgroundColor = "blue";
button.style.color = "white";
});
```

يغير هذا المثال لون الزر إلى الأزرق ويجعل النص أبيض عند الضغط عليه.

ويمكن استخدام أحداث أخرى لتنفيذ تأثيرات مختلفة، مثل إظهار قائمة عند مرور المؤشر فوق عنصر، أو عرض وصف مختصر لصورة.

### ثانيًا: التفاعل مع لوحة المفاتيح

تُستخدم لوحة المفاتيح في كتابة النصوص والبحث وإدخال البيانات. ويمكن لـ JavaScript الاستجابة للأحداث المرتبطة بالمفاتيح لتنفيذ أوامر محددة.

ومن الأمثلة على ذلك اكتشاف الضغط على مفتاح معين أثناء استخدام الصفحة:

```
document.addEventListener("keydown", function(event) {
if (event.key === "Enter") {
console.log("تم الضغط على مفتاح Enter");
}
});
```

في هذا المثال، يستمع البرنامج إلى حدث keydown، ثم يفحص المفتاح الذي تم الضغط عليه باستخدام الخاصية event.key. فإذا كان المفتاح هو Enter، يكتب البرنامج رسالة في وحدة التحكم الخاصة بالمتصفح.

### أهمية التفاعل مع الماوس ولوحة المفاتيح

يساعد هذا النوع من التفاعل على جعل المواقع أكثر سهولة ومرونة. فالمستخدم يستطيع تنفيذ الأوامر باستخدام الماوس أو لوحة المفاتيح وفقًا لما يناسبه.

كما يمكن الاستفادة من اختصارات لوحة المفاتيح في التطبيقات المتقدمة، ومن تأثيرات الماوس في تحسين وضوح الأزرار والعناصر القابلة للاستخدام.

ومع ذلك، ينبغي تصميم التفاعل بطريقة لا تربك المستخدم، مع مراعاة احتياجات الأشخاص الذين يعتمدون على لوحة المفاتيح بدلًا من الماوس، والتأكد من أن العناصر المهمة يمكن الوصول إليها بسهولة.

## التفاعل مع النماذج الإلكترونية

### مفهوم النماذج الإلكترونية

النماذج الإلكترونية هي أجزاء من صفحات الويب تسمح للمستخدم بإدخال البيانات وإرسالها، مثل نماذج تسجيل الدخول وإنشاء الحسابات والتواصل مع إدارة الموقع وطلب المنتجات.

وتساعد JavaScript في تحسين طريقة التعامل مع هذه النماذج، حيث يمكنها فحص البيانات التي يدخلها المستخدم، وعرض رسائل توضيحية، ومنع إرسال بعض البيانات غير الصحيحة قبل انتقالها إلى الخادم.

### التحقق من البيانات

من أهم استخدامات JavaScript التحقق من صحة البيانات المدخلة. فعلى سبيل المثال، يمكن التأكد من أن المستخدم لم يترك حقل الاسم فارغًا.

```
<form id="myForm">
<input id="userName" type="text" placeholder="اكتب اسمك">
<button type="submit">إرسال</button>
</form>
<p id="result"></p>
const form = document.getElementById("myForm");
const nameInput = document.getElementById("userName");
const result = document.getElementById("result");
form.addEventListener("submit", function(event) {
event.preventDefault();
if (nameInput.value.trim() === "") {
result.textContent = "من فضلك اكتب اسمك أولًا";
} else {
result.textContent = "تم إدخال الاسم بنجاح";
```

### شرح المثال

عندما يحاول المستخدم إرسال النموذج، يتم تنفيذ الوظيفة المرتبطة بحدث submit.وتمنع التعليمة event.preventDefault()الإرسال الافتراضي للنموذج في هذا المثال، حتى يستطيع البرنامج فحص البيانات وعرض النتيجة.

بعد ذلك، يتحقق البرنامج مما إذا كان حقل الاسم فارغًا. فإذا كان فارغًا، تظهر رسالة تطلب من المستخدم كتابة اسمه. أما إذا كان يحتوي على نص، فتظهر رسالة تؤكد إدخال الاسم.

### تطبيقات عملية أخرى

يمكن استخدام JavaScript في:

- التأكد من إدخال البريد الإلكتروني بصيغة مناسبة.
- التحقق من أن كلمة المرور تستوفي الشروط المطلوبة.
- إظهار عدد الأحرف المتبقية في مربع الكتابة.
- عرض رسالة تأكيد قبل تنفيذ عملية مهمة.
- تحديث بعض البيانات المعروضة بناءً على اختيارات المستخدم.

ويجب التأكيد على أن التحقق باستخدام JavaScript داخل المتصفح لا يغني عن التحقق من البيانات على الخادم؛ لأن المستخدم يستطيع تجاوز بعض الفحوصات الموجودة في الصفحة. لذلك يجب تطبيق ضوابط الأمان المناسبة لحماية الموقع وبيانات المستخدمين.

## دور JavaScript في تحسين تجربة المستخدم

### مفهوم تجربة المستخدم

تجربة المستخدم هي الانطباع العام الذي يتكون لدى الشخص أثناء استخدام موقع أو تطبيق. وتشمل سهولة الوصول إلى المعلومات، وسرعة تنفيذ المهام، ووضوح التعليمات، ومدى استجابة الموقع للتصرفات المختلفة.

وتساعد JavaScript على تحسين تجربة المستخدم من خلال جعل الموقع يستجيب بطريقة واضحة ومباشرة للأوامر التي ينفذها الزائر.

### أهم وسائل تحسين تجربة المستخدم

**١. عرض الرسائل الفورية**

يمكن للموقع إظهار رسالة تؤكد نجاح عملية ما أو توضح وجود خطأ. فعند حفظ البيانات، يمكن عرض رسالة تفيد بأن العملية تمت بنجاح.

**٢. تحديث المحتوى دون إعادة تحميل الصفحة بالكامل**

يمكن استخدام JavaScript مع تقنيات الويب المناسبة لتحديث جزء من الصفحة بدلًا من إعادة تحميلها كاملة في كل مرة. ويساعد ذلك على جعل بعض العمليات أكثر سلاسة.

**٣. إنشاء القوائم التفاعلية**

يمكن إظهار قائمة عند الضغط على زر معين، ثم إخفاؤها عند الضغط مرة أخرى. وتفيد هذه الطريقة في تنظيم المحتوى، خاصة في المواقع التي تحتوي على أقسام كثيرة.

**٤. تحسين البحث**

يمكن للموقع الاستجابة لما يكتبه المستخدم داخل مربع البحث، وتقديم اقتراحات مناسبة أو تصفية العناصر المعروضة وفقًا للكلمات المدخلة.

**٥. إنشاء النوافذ المنبثقة**

يمكن إظهار نافذة تحتوي على معلومات أو تأكيد لعملية معينة. وينبغي استخدام هذه النوافذ باعتدال حتى لا تصبح مزعجة للمستخدم.

**٦. توفير التغذية الراجعة**

التغذية الراجعة هي المعلومات التي يحصل عليها المستخدم بعد تنفيذ إجراء معين. فعندما يضغط على زر، من المفيد أن يعرف هل استجاب الموقع للضغط أم لا، وما الخطوة التالية التي يمكنه القيام بها.

### مثال من الحياة اليومية

عند استخدام متجر إلكتروني، يضغط المستخدم على زر «أضف إلى السلة». يمكن أن تستخدم الصفحة JavaScript لتحديث عدد المنتجات الموجودة في السلة، وعرض رسالة تؤكد إضافة المنتج. وبهذه الطريقة، يستطيع المستخدم معرفة نتيجة الإجراء مباشرة، بدلًا من الانتقال إلى صفحة جديدة في كل مرة.

إن الهدف الأساسي من التفاعل ليس إضافة الحركة إلى الموقع فقط، بل جعل استخدامه أكثر وضوحًا وسهولة، مع الحفاظ على سرعة الأداء وعدم تشتيت انتباه الزائر.

## تطبيق عملي ومثال متكامل

## إنشاء صفحة ويب تفاعلية بسيطة

يمكن تطبيق المفاهيم السابقة من خلال إنشاء صفحة صغيرة تحتوي على عنوان، وزر، ومربع نص، ورسالة تظهر نتيجة لتفاعل المستخدم.

يهدف المثال إلى توضيح كيفية جمع HTML و CSSو JavaScriptفي تطبيق واحد.

## كود التطبيق

يمكن حفظ الكود التالي في ملف باسم index.html، ثم فتح الملف باستخدام متصفح الويب:

```
<!DOCTYPE html>
<html lang="ar" dir="rtl">
<head>
<meta charset="UTF-8">
<title> صفحة تفاعلية</title>
<style>
body {
font-family: Arial, sans-serif;
text-align: center;
margin-top: 60px;
background-color: #f2f5f9;
}
input, button {
padding: 10px;
margin: 8px;
font-size: 16px;
}
button {
background-color: #164b8c;
color: white;
border: none;
cursor: pointer;
}
</style>
</head>
<body>
<h1> مرحبًا بك</h1>
<input id="name" type="text"
placeholder=" اكتب اسمك هنا">
<button id="showButton"> عرض الترحيب</button>
<p id="message"></p>
<script>
const button = document.getElementById("showButton");
const input = document.getElementById("name");
const message = document.getElementById("message");
button.addEventListener("click", function() {
```

```
const name = input.value.trim();
if (name === "") {
  message.textContent = "من فضلك اكتب اسمك.";
} else {
  message.textContent = "أهلًا بك يا " + name;
}
});
</script>
</body>
</html>
```

## كيف يعمل التطبيق؟

في البداية، تنشئ HTML العناصر الأساسية للصفحة، وهي العنوان وحقل إدخال الاسم والزر ومكان عرض الرسالة.

بعد ذلك، تحدد CSS مظهر الصفحة، وتضبط محاذاة النصوص وألوان الزر وحجم العناصر.

أما JavaScript فتتولى الجزء التفاعلي. فهي تصل إلى العناصر باستخدام document.getElementById()، ثم تربط الزر بحدث الضغط.

عندما يضغط المستخدم على الزر، يقرأ البرنامج الاسم الذي كتبه، ويحذف المسافات الزائدة من بدايته ونهايته باستخدام trim(). فإذا لم يكتب المستخدم اسمًا، تظهر رسالة تطلب منه إدخاله. وإذا كتب اسمه، تظهر رسالة ترحيب تحتوي عليه.

## ماذا نتعلم من المثال؟

يوضح هذا التطبيق أن إنشاء موقع تفاعلي لا يتطلب دائمًا أكوادًا معقدة. فباستخدام عدد قليل من التعليمات، يمكن جعل الصفحة تستقبل بيانات المستخدم، وتفحصها، وتعرض نتيجة مناسبة.

ويمكن تطوير المثال مستقبلًا بإضافة زر لمسح الاسم، أو تغيير لون الرسالة، أو إنشاء قائمة بأسماء المستخدمين، أو إضافة نموذج تسجيل بسيط.

ويُعد هذا النوع من التطبيقات خطوة أولى جيدة لفهم طريقة بناء المواقع التفاعلية، قبل الانتقال إلى المشروعات الأكبر التي تستخدم قواعد البيانات والاتصال بالخوادم.

## التحديات والخاتمة والمراجع

### التحديات التي تواجه استخدامJavaScript

على الرغم من أهمية JavaScript في تطوير المواقع، فإن استخدامها قد يرتبط ببعض التحديات التي ينبغي للمبرمج الانتباه إليها.

### أولًا: الأخطاء البرمجية

قد يؤدي وجود خطأ في الكود إلى عدم استجابة الزر أو توقف وظيفة معينة. ولذلك يحتاج المبرمج إلى اختبار الموقع واستخدام أدوات المطور في المتصفح لاكتشاف الأخطاء وإصلاحها.

### ثانيًا: الأداء والسرعة

قد يؤدي استخدام عدد كبير من الوظائف أو تنفيذ عمليات ثقيلة أثناء كل تفاعل إلى بطء الصفحة. لذلك ينبغي كتابة كود منظم وتجنب العمليات غير الضرورية.

### ثالثًا: الأمان

لا ينبغي الاعتماد على الفحص الذي يجري داخل المتصفح وحده لحماية البيانات. فمن المهم التحقق من البيانات على الخادم، واستخدام وسائل الحماية المناسبة عند التعامل مع الحسابات والمعلومات الشخصية.

### رابعًا: سهولة الاستخدام

قد يؤدي الإفراط في الحركات والنوافذ المنبثقة إلى إزعاج المستخدم. ولذلك يجب أن يكون التفاعل مفيدًا وواضحًا، وأن يخدم هدف الصفحة بدلًا من تشتيت الانتباه.

### خامسًا: التوافق مع الأجهزة والمتصفحات

ينبغي اختبار الموقع على المتصفحات والأجهزة المختلفة، والتأكد من أن عناصره التفاعلية تعمل بصورة مناسبة، سواء استخدم الزائر الماوس أو شاشة اللمس أو لوحة المفاتيح.

### الخاتمة

في نهاية هذا البحث، يتضح أن JavaScript تؤدي دورًا أساسيًا في جعل مواقع الويب قادرة على الاستجابة لتصرفات المستخدمين. فمن خلالها يستطيع المبرمج التعامل مع الضغط على الأزرار، وإدخال النصوص، وإرسال النماذج، وتغيير محتوى الصفحة، وعرض الرسائل التي تساعد المستخدم على فهم نتيجة كل إجراء.

كما تبين أن مفهوم الأحداث يمثل أساسًا مهمًا لفهم التفاعل، حيث يحدد الحدث التصرف الذي وقع، بينما يحدد معالج الحدث التعليمات التي ستنفذ استجابةً له.

وقد أوضحت الأمثلة العملية أن الجمع بين HTML و CSSو JavaScriptيسمح بإنشاء صفحات تجمع بين البنية المنظمة والتصميم الجيد والتفاعل المباشر.

ومع استمرار تطور تقنيات الويب، تظل JavaScript من الأدوات المهمة التي تساعد المطورين على بناء تجارب رقمية أكثر مرونة وسهولة. ولذلك فإن تعلم أساسياتها وفهم طريقة عمل الأحداث يمثلان خطوة مهمة لكل من يرغب في دخول مجال تصميم وتطوير مواقع الويب.

# المراجع


١. MDN Web Docs، **مقدمة إلى الأحداث في** JavaScript، Mozilla Developer Network.
https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Events
٢. MDN Web Docs، **مرجع الطريقة** addEventListener().
https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener
٣. MDN Web Docs، **أحداث نموذج كائن المستند** (DOM Events).
https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Events
٤. W3Schools، **دروس** JavaScript **والأحداث في صفحات الويب**.
https://www.w3schools.com/js/js_events.asp
٥. JavaScript.info، **دليل أساسيات** JavaScript **والأحداث في المتصفح**.
https://javascript.info/events